대덕고 튜터링 1차시

강사 소개

KAIST GoN

주강사 : 선우준

qwilight@kaist.ac.kr

보조 강사 : 전도연, 유영석

1.Python

Python?

인터프리터 언어

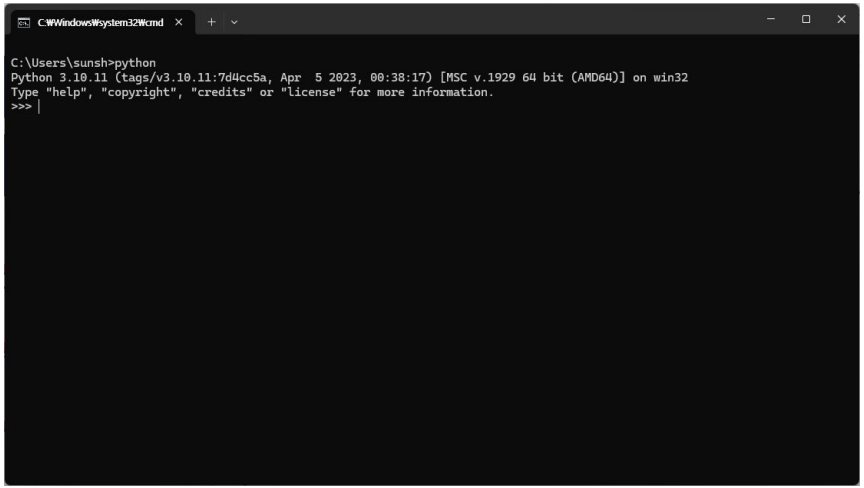
문법이 직관적이고 간단함

쉽게 새로운 모듈을 설치하고 사용 가능함

다른 언어와 괴리가 존재할 수 있음

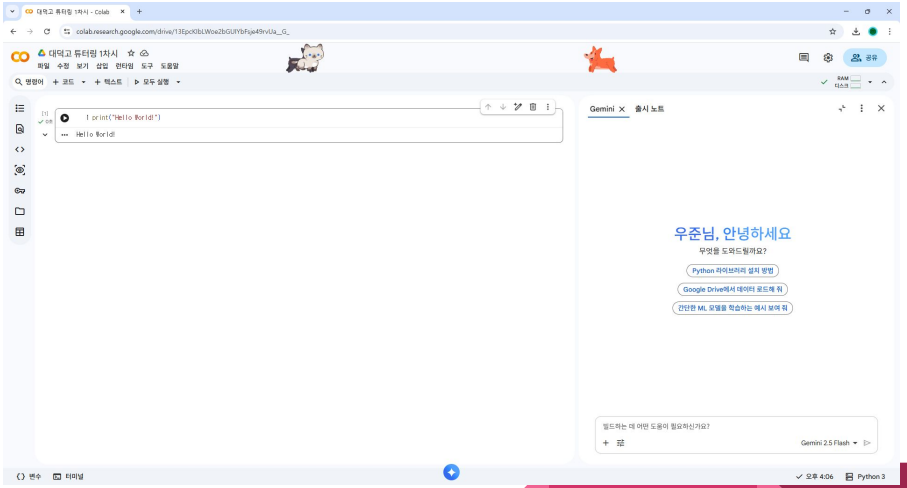
어디서 돌리냐?

직접 설치



내 컴퓨터의 자원을 자유롭게 활용 가능

Colab등을 활용하기



고성능의 자원 활용
쉬운 AI 연동

AI 활용

코드 쓸때 AI를 활용하면 노력을 덜들이고 작성할 수 있다

AI를 쓰기전에 AI가 뭐 하는지 정도는 알아가기 위해 기초는 배우자

기본 툴 설치

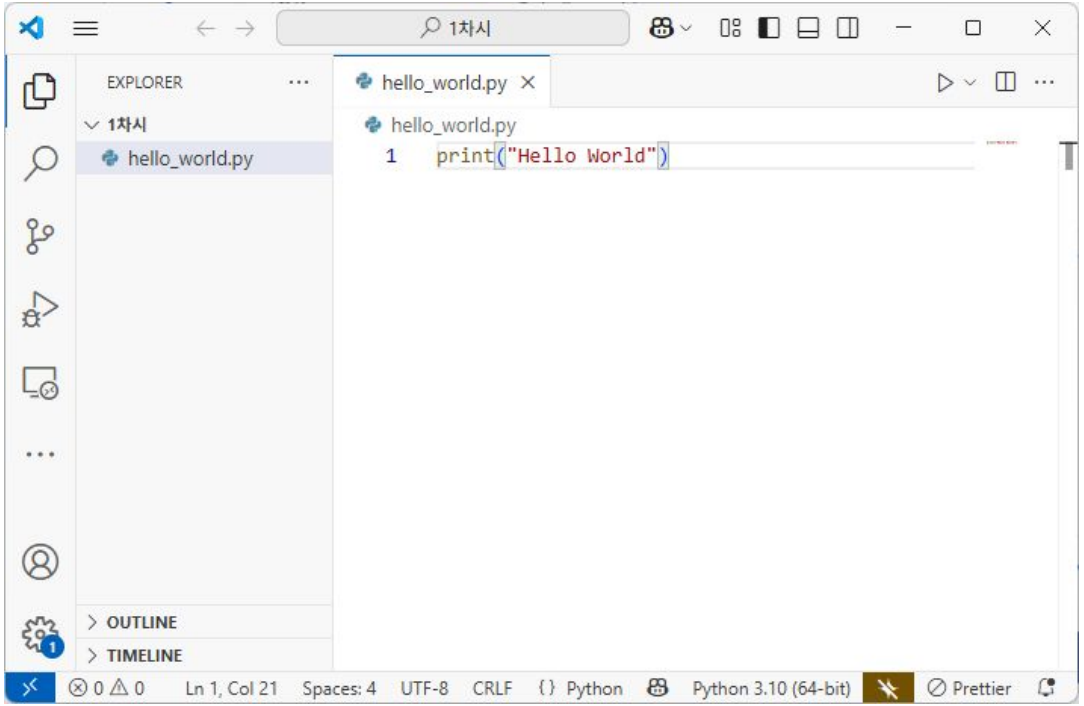
Python 런타임

https://www.python.org/downloads/
2026/04/09 기준 3.14.4 강의 자료는 3.10을 이용해 작성하였지만..

Visual Studio Code(텍스트 에디터)

https://code.visualstudio.com/download
다양한 언어 개발에 활용 가능

Hello World!

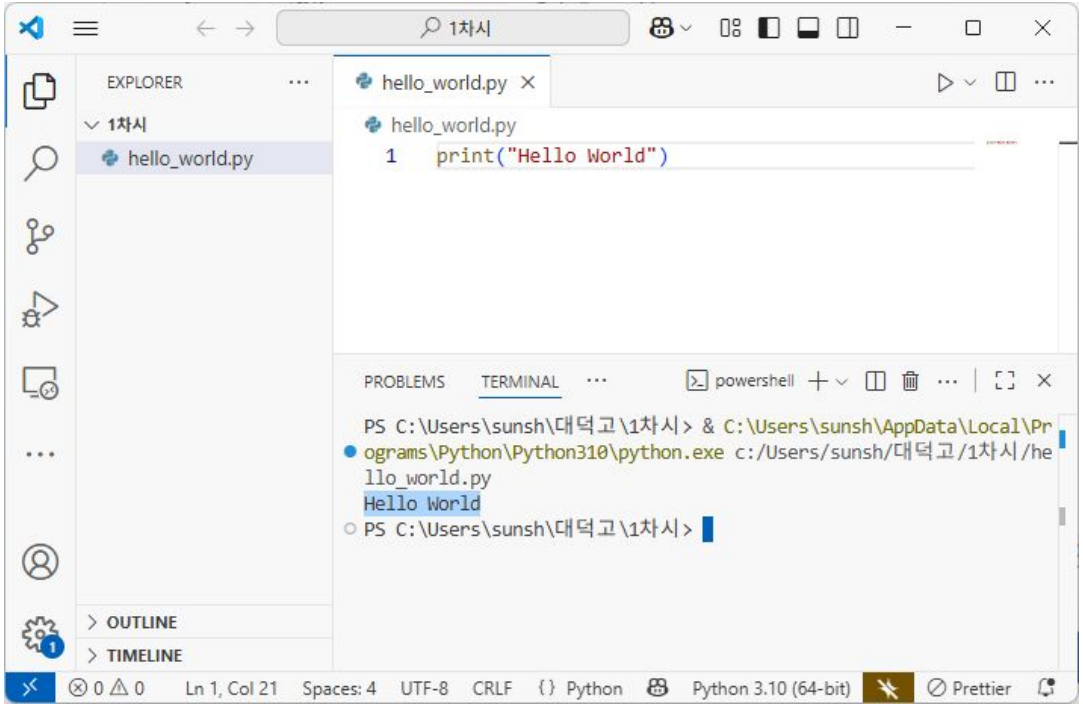


VSCode를 열어 .py 확장자로 끝나는 파일 하나를 만들고

print(“Hello World”)

를 입력해보자

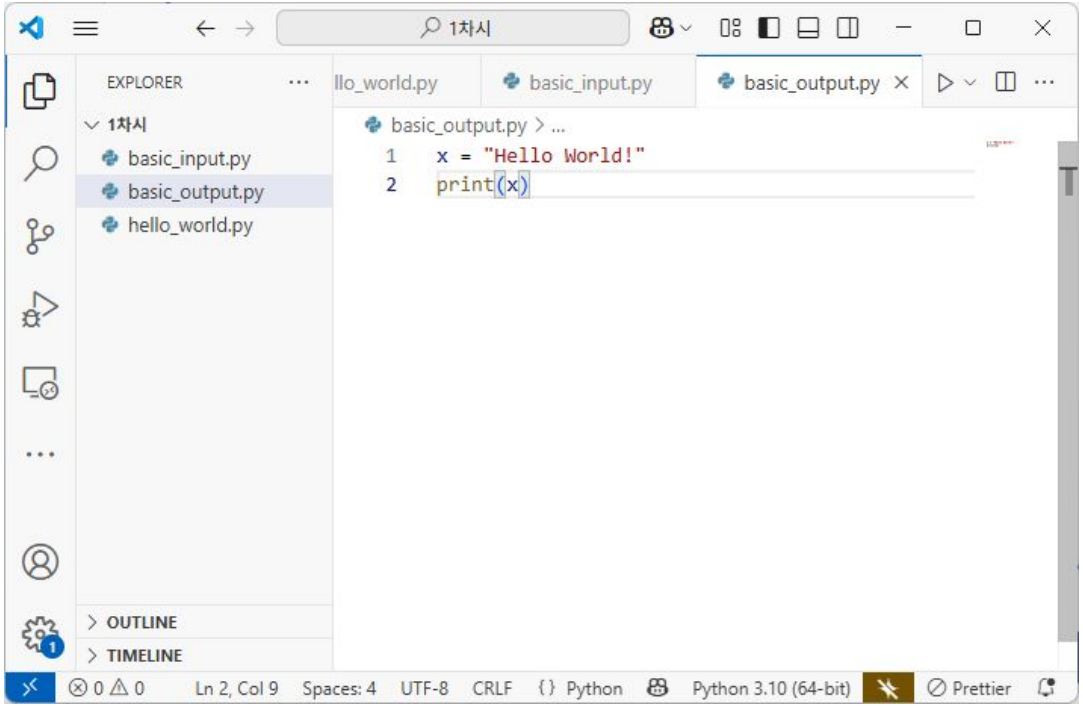
실행하기



우측 상단의 ▷를 눌러 실행하거나

터미널에서
python hello_world.py

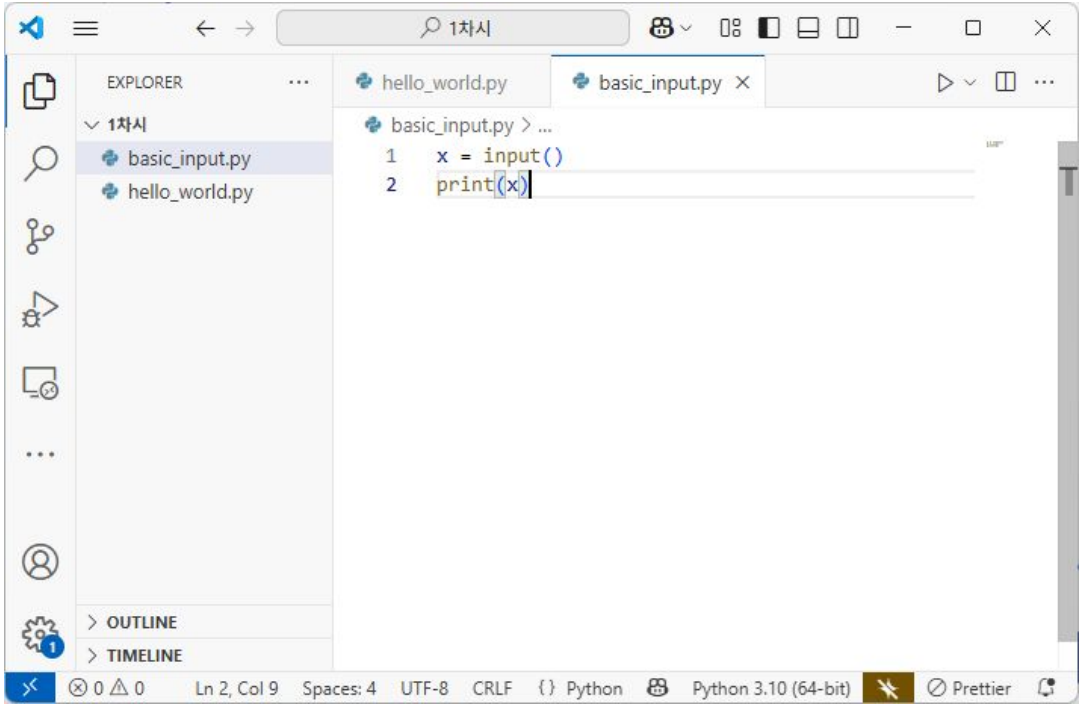
출력



파이썬에서 출력은
print() 함수를 이용한다

print() 함수의 다양한 활용은
나중에..

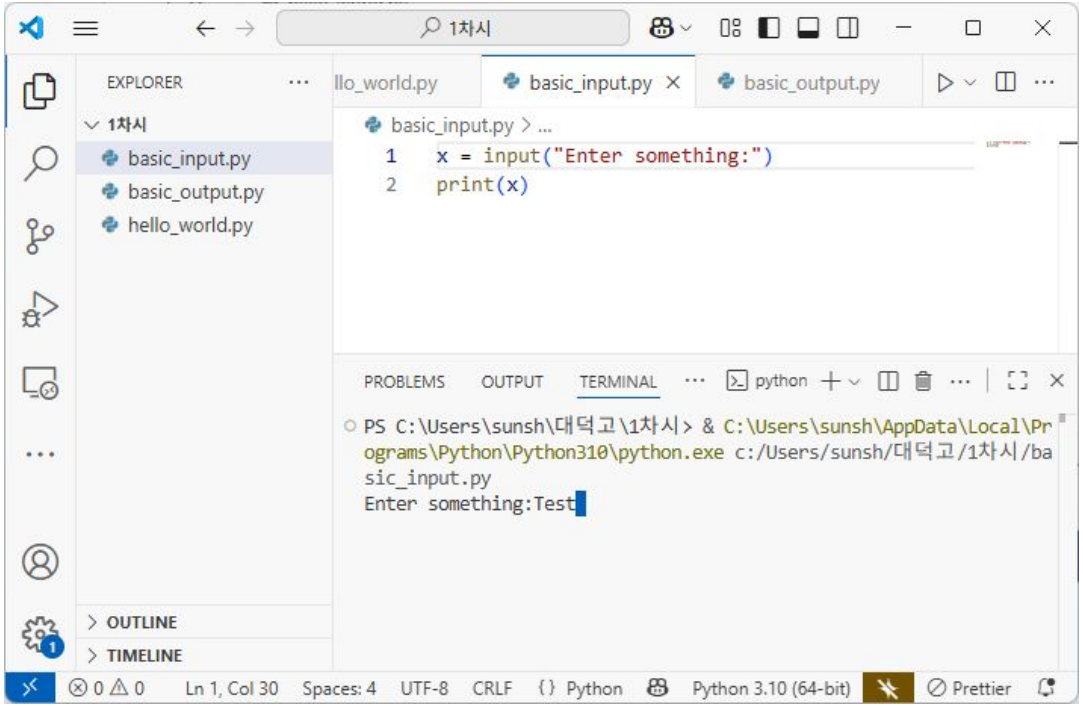
입력



입력은 어떻게 할까?

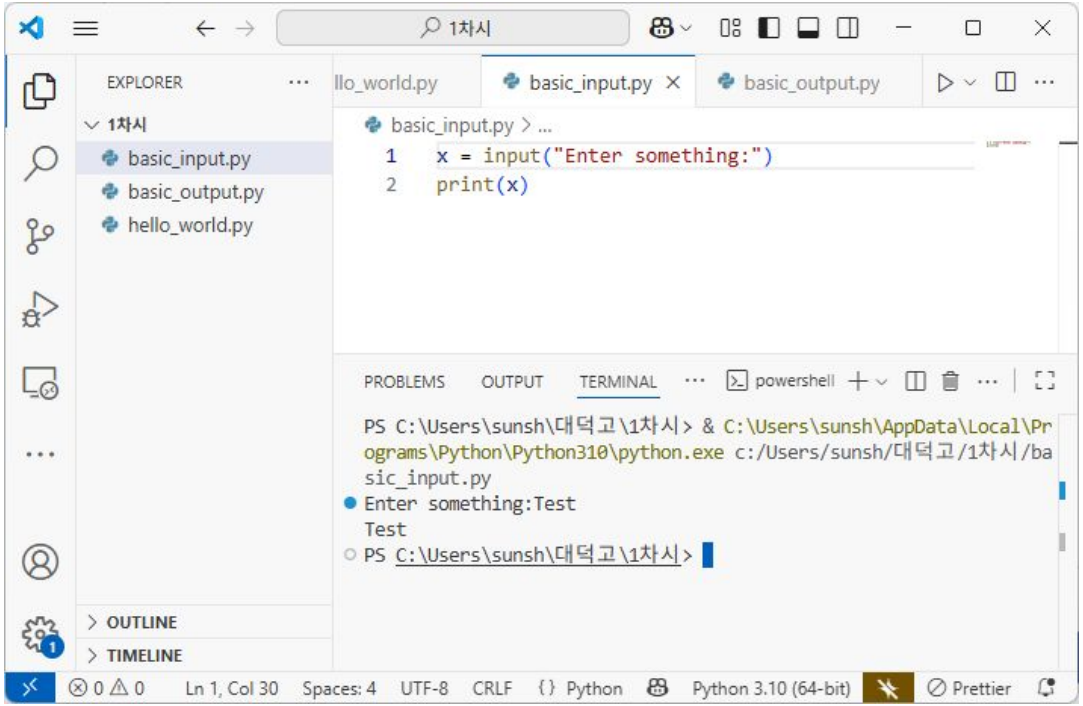
파이썬에서 입력은
input() 함수를 이용한다

입력



input 함수의 매개변수에 입력받을 내용을 넣을 수도 있다.

입력


1차시
EXPLORER
1차시
basic_input.py
basic_output.py
hello_world.py
llo_world.py
basic_input.py
basic_output.py
basic_input.py > ...
1 x = input("Enter something:")
2 print(x)
PROBLEMS
OUTPUT
TERMINAL
powershell
PS C:\Users\sunsh\대덕고\1차시> & C:\Users\sunsh\AppData\Local\Programs\Python\Python310\python.exe c:/Users/sunsh/대덕고/1차시/basic_input.py
Enter something:Test
Test
PS C:\Users\sunsh\대덕고\1차시>
OUTLINE
TIMELINE
0 0
Ln 1, Col 30
Spaces: 4
UTF-8
CRLF
Python
Python 3.10 (64-bit)
Prettier

변수

{변수 이름} = {변수 값} 형태로 저장함

변수에는 Type가 있음

Python은 type(변수)를 통해 type확인이 가능

단일한 값을 저장하는 변수와(int, float, bool, string)
여러 값을 저장하는 변수가 존재함(string, list, set, dict)

basic_variables.py > ...

```python
# 수 : number
a = 1 # int
b = 2.2 # float

# 문자열 : string
c = "Hello World"

# Bool(참 / 거짓) : bool
d = True # or False

# 리스트 : list
e = [1,2,3]

# 튜플 : tuple
f = (1,2,3)

# 집합 : set
g = {1,2,3}

# 사전 : dict
h = {"one": 1, "two": 2, "three": 3}
```

variables_type.py > ...

```python
a = 1
b = "Hello World"

print(type(a))
print(type(b))

a = "String!!"

print(type(a))
```

PROBLEMS TERMINAL ··· powershell

```
ograms\Python\Python310\python.exe c:/Users/sunsh/대덕고/1차시/va
riables_type.py
<class 'int'>
<class 'str'>
<class 'str'>
PS C:\Users\sunsh\대덕고\1차시>
```

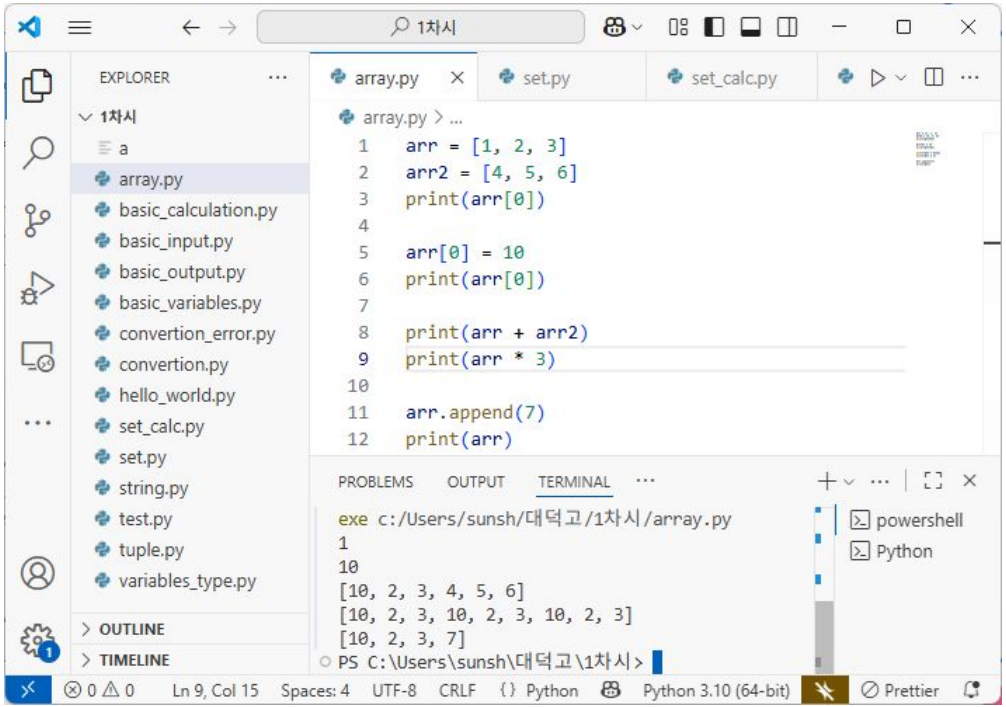
여러 값을 저장하는 변수

처음은 0부터 센다

n번째 값은 {변수이름}[n] 형태로 접근 가능

정해진 길이를 넘어 접근하면 IndexError 발생!

배열(array)

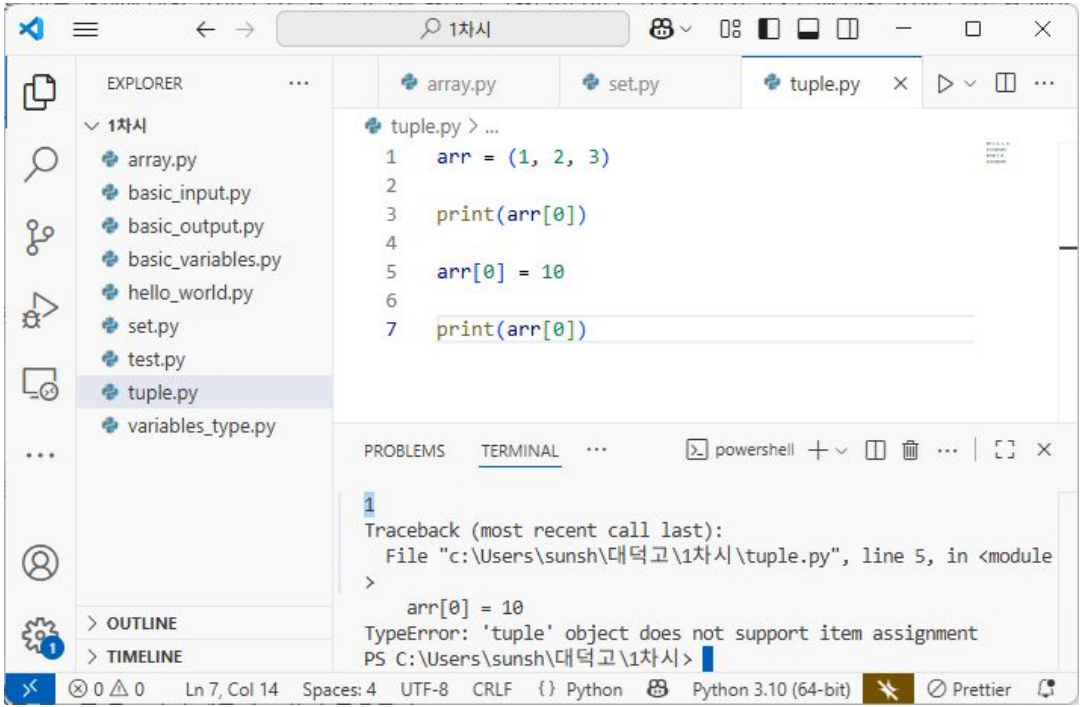


값 변경이 가능함

+로 배열을 붙여서 돌려줄 수 있음

.append()로 원소 추가 가능함

튜플(tuple)



리스트와 비슷함

생성후 값 변경이 불가능함

산술 연산(수)

더하기, 빼기, 곱하기, 나누기등 사칙연산

// 몫 연산

% 나머지 연산

** 제곱 연산

산술 연산 예시

basic_calculation.py > ...

```python
a = 32
b = 2

print(a + b)   =   34
print(a - b)   =   30
print(a * b)   =   64
print(a / b)   =   16.0
print(a // b)  =   16
print(a % b)   =   0
print(a ** b)  =   1024
```

수학에서 사용하는 연산과 같다

/과 // 사용할 때 구분

a, b가 양수일때 a // b = int(a / b)

변환 연산 / 문자열 연산

convertion.py

```python
print(int("32"))

print(float("23.0"))

print(str(32))
```

문자열 / 수 간에
type 변환이 가능

PROBLEMS OUTPUT TERMINAL ···

```
PS C:\Users\sunsh\대덕고\1차시> & C:\Users\sunsh
\AppData\Local\Programs\Python\Python310\python.
exe c:/Users/sunsh/대덕고/1차시/convertion.py
32
23.0
32
PS C:\Users\sunsh\대덕고\1차시>
```

ces: 4 UTF-8 CRLF {} Python Python 3.10 (64-bit)

convertion_error.py

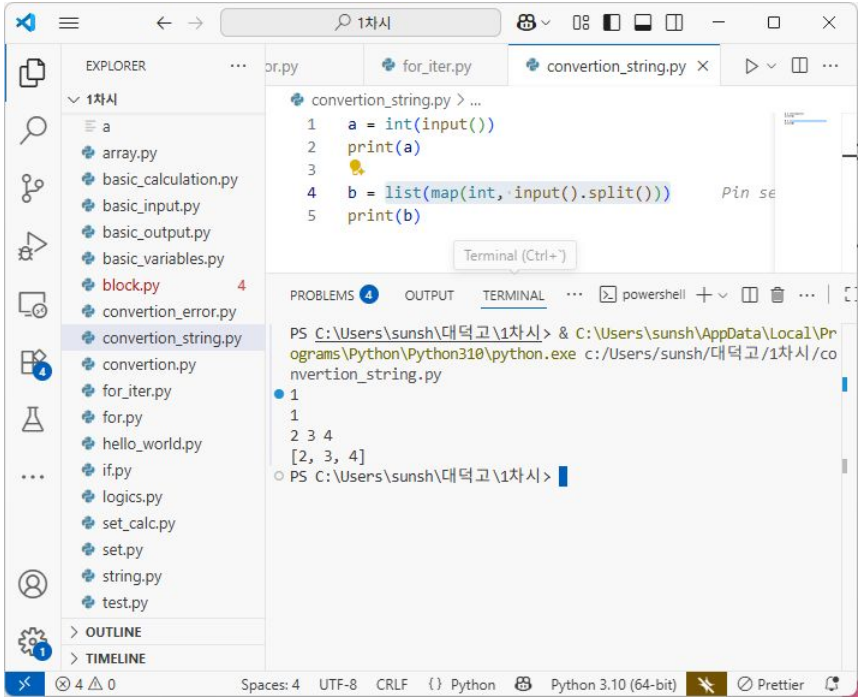
```python
print(int("32sdf"))
```

당연히 이상한거
넣으면 error

PROBLEMS OUTPUT TERMINAL ···

```
.py
Traceback (most recent call last):
  File "c:\Users\sunsh\대덕고\1차시\convertion_e
rror.py", line 1, in <module>
    print(int("32sdf"))
ValueError: invalid literal for int() with base
10: '32sdf'
PS C:\Users\sunsh\대덕고\1차시>
```

변환 연산 / 문자열 연산



int(input()) 으로 정수 하나 입력받기

list(map(int, input().split())) 으로 여러 정수 입력받기

문자열 연산

string.py > ...

```python
a = "abc"

b = "adf"

print(a + b)
print(a * 3)
print(b * 2)
```

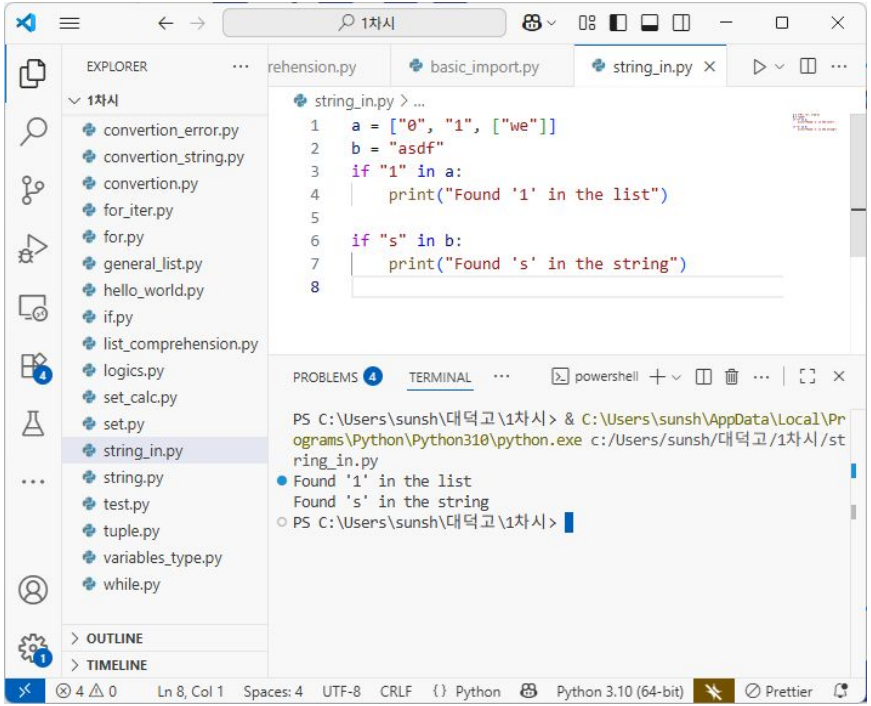
PROBLEMS OUTPUT TERMINAL ···

```
PS C:\Users\sunsh\대덕고\1차시> & C:\Users\sunsh
\AppData\Local\Programs\Python\Python310\python.
exe c:/Users/sunsh/대덕고/1차시/string.py
abcadf
abcabcabc
adfadf
PS C:\Users\sunsh\대덕고\1차시>
```

배열과 사실상 동일하게 사용할 수 있음

append를 제외한 +, *, []를 이용한 인덱싱

문자열 연산



in 키워드를 사용하면 배열이나 문자열에 특정 문자열이 들어가 있는지 확인이 가능하다

논리 연산

조건이 맞으면 True, 아니면 False

== 같은가?
!= 다른가?
> 왼쪽이 오른쪽보다 큼?
>= 왼쪽이 오른쪽보다 크거나 같음?
< 왼쪽이 오른쪽보다 작음?
<= 왼쪽이 오른쪽보다 작거나 같음?

not A 부정
A and B 그리고
A or B 또는

logics.py

```python
print(1 == 2)
print(1 == 1)
print(1 != 2)
print(1 != 1)

print(1 < 2)
print(1 > 2)
print(1 <= 1)
print(1 >= 1)

print(not True)
print(True and False)
print(True or False)
```

PROBLEMS OUTPUT TERMINAL ···

```
PS C:\Users\sunsh\대덕고\1차시> & C:\Users\sunsh
\AppData\Local\Programs\Python\Python310\python.
exe c:/Users/sunsh/대덕고/1차시/logics.py
False
True
True
False
True
False
True
True
False
False
True
PS C:\Users\sunsh\대덕고\1차시>
```

추가로 비트 연산자가 있으나 생략..

분기문, 반복문을 들어가기에 앞서

block.py

```
1  print("1")
2      print("2")
```

PROBLEMS 4 OUTPUT TERMINAL powershell

```
PS C:\Users\sunsh\대덕고\1차시> & C:\Users\sunsh\AppData\Local\Programs\Python\Python310\python.exe c:/Users/sunsh/대덕고/1차시/block.py
  File "c:\Users\sunsh\대덕고\1차시\block.py", line 2
    print("2")
IndentationError: unexpected indent
PS C:\Users\sunsh\대덕고\1차시>
```

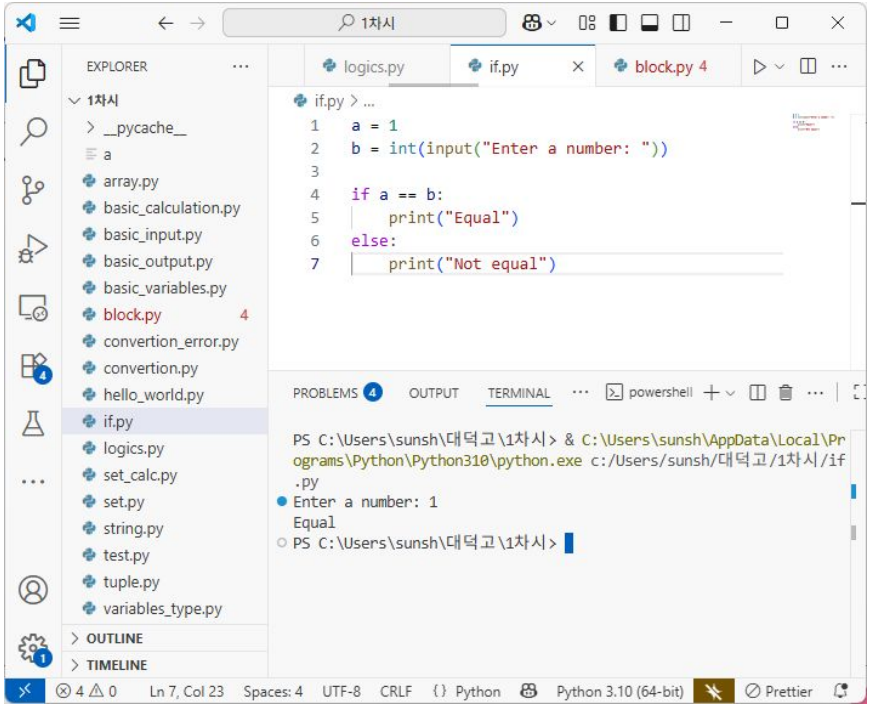
python은 코드 블럭(특정한 동작을 위한 코드의 모음)을 Enter를 제외한 공백문자(tab('\t'), 스페이스(' '))로 구분한다(indent)

보통 스페이스 4개를 블럭 구분으로 사용하며 기본적으로 에디터에서tab을 누르면 띄어쓰기 4개가 삽입된다.

사실 스페이스 개수는 상관없다

들여쓰기로 코드를 구분하므로 의미없는 들여쓰기는 왼쪽과 같이 에러로 취급된다

분기문(if)



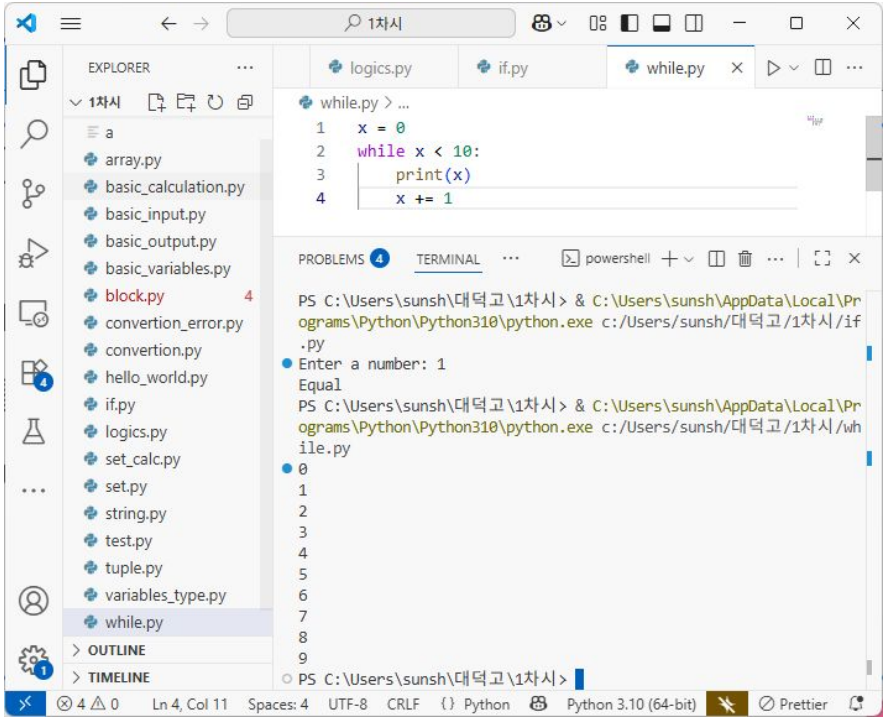
if {조건} :(콜론) 형태로 구성

if문 아래 나오는 블럭에 “조건이 맞을때” 실행할 내용 작성

else문 아래 나오는 블럭에는 “조건이 틀릴때” 실행할 내용 작성

elif를 이용해 중첩 가능

반복문(While)



while {조건} :(콜론) 형태

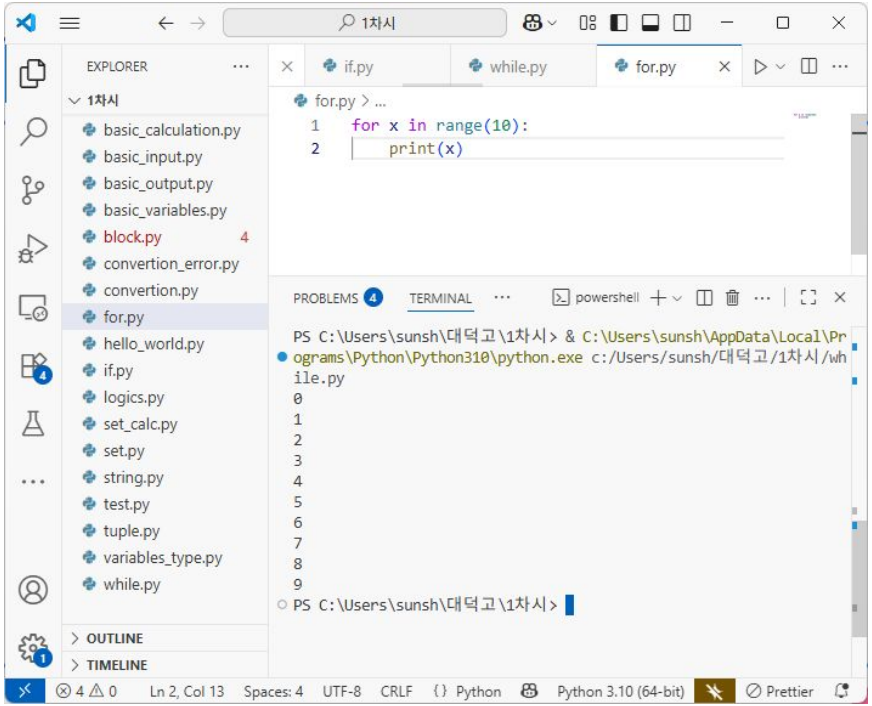
조건이 True인 동안 반복

while True:를 이용해 무한반복이 가능하다

break를 이용해 while문 탈출

continue를 이용해 while문 앞으로 돌아갈 수 있음

반복문(for)



for {원소} in {반복할 리스트}: 형태

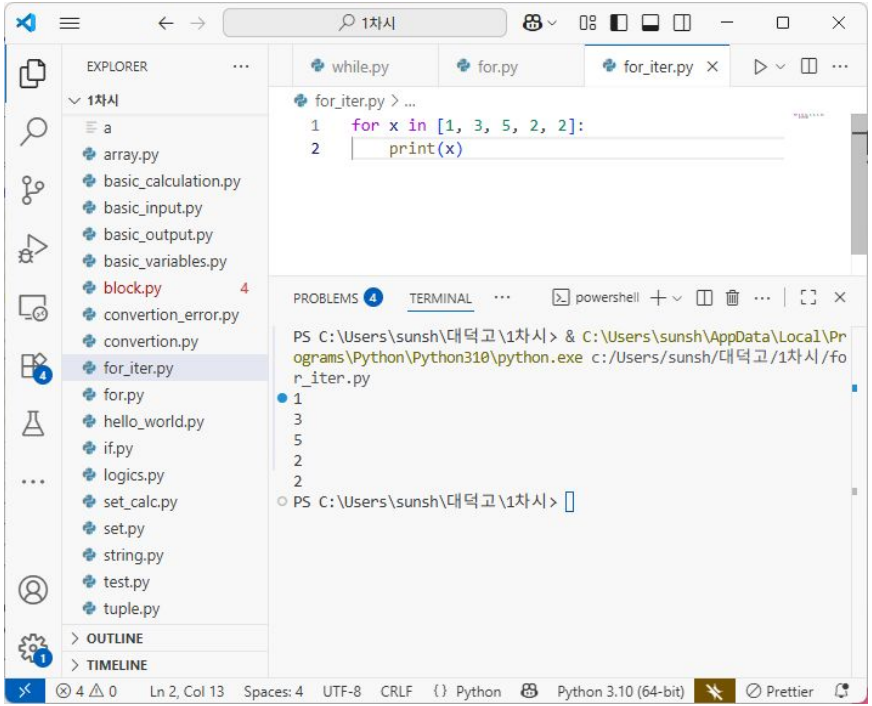
range(10) : 0부터 10개의 수를 반환

range(0, 10): 0부터 10 미만의 수를 반환

range(0, 10, 1): 0부터 10보다 작을때까지 1씩 증가하며 반환

range(10, 0, -1): 10부터 0보다 클때까지 1씩 감소하며 반환

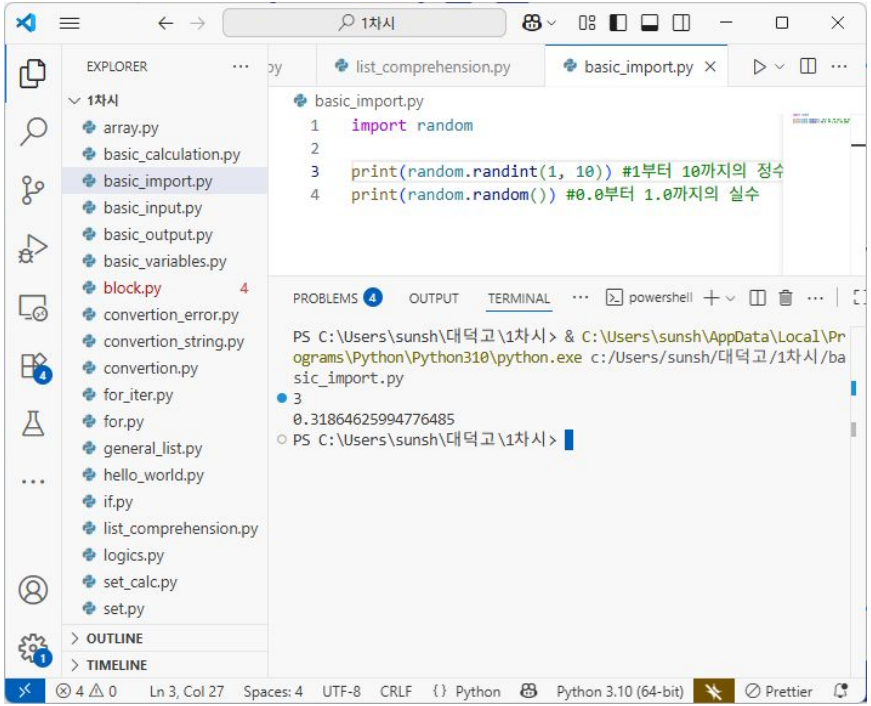
반복문(for)



for {원소} in {반복할 리스트}: 형태

일반 리스트를 in 뒤에 쓰면 하나씩 뽑아 사용할 수 있다.

import 사용법



import {모듈 이름}으로 모듈을 import할 수 있음

원하는 모듈을 자유롭게 설치 및 import 가능

random모듈에는 randint, random등 함수가 존재

randint : 이상 이하(끔찍하다)

random : 0.0, 1.0사이 실수 -> 응용

import 사용법

```
C:\Users\sunsh>pip install tqdm
Requirement already satisfied: tqdm in c:\users\sunsh\appdata\local
Requirement already satisfied: colorama in c:\users\sunsh\appdata\l
m tqdm) (0.4.6)

[notice] A new release of pip is available: 25.0 -> 26.0.1
[notice] To update, run: python.exe -m pip install --upgrade pip

C:\Users\sunsh>
```

```
C:\Users\sunsh>python
Python 3.10.11 (tags/v3.10.11:7d4cc5a, Apr  5 2023, 00:38:17) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> import tqdm
>>> from tqdm import tqdm
>>> |
```

```
C:\Users\sunsh>python
Python 3.10.11 (tags/v3.10.11:7d4cc5a, Apr  5 2023, 00:38:17) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> from tqdm import tqdm
>>> x = [i for i in tqdm(range(1,1000))]
100%|[redacted]| 999/999 [00:00<?, ?it/s]
>>> |
```

pip install {패키지 이름}

pip install --r requirements.txt

실습

1. 두 정수 입력받아 사칙연산 결과 출력하기
2. 여러 정수 입력받아 리스트로 만들고, 뒤집어서 출력하기
3. 피보나치 수열
4. 가위바위보 게임 만들기

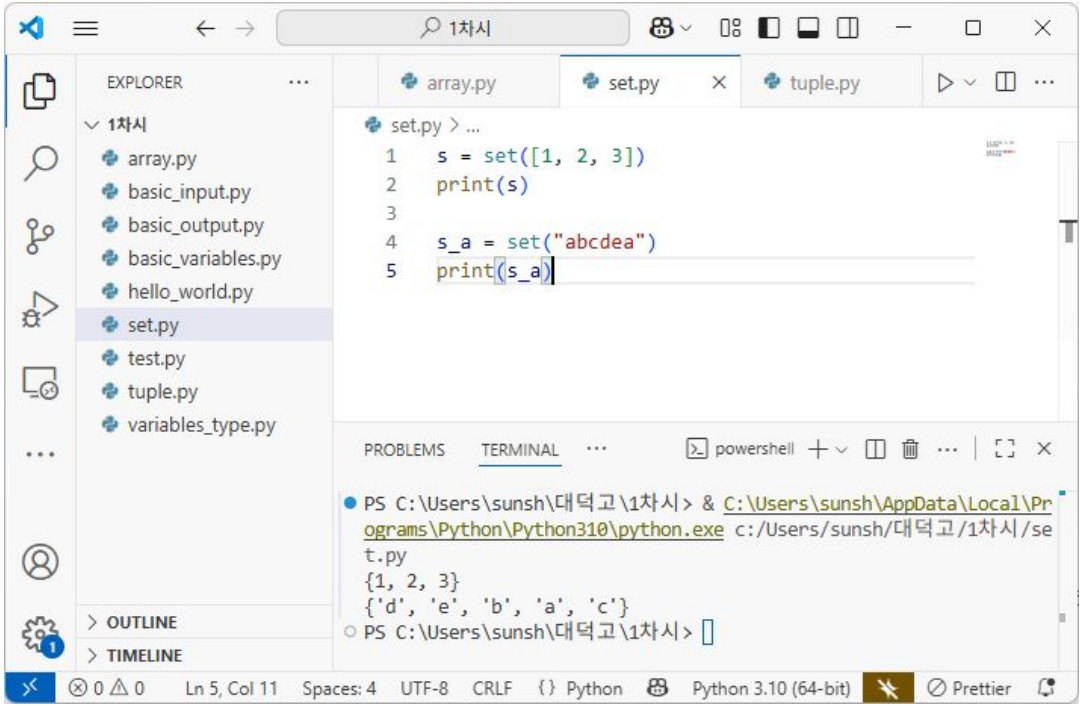
내용 추가

print 문

print({출력할 내용1}, {출력할 내용2}) 로 여러개 출력

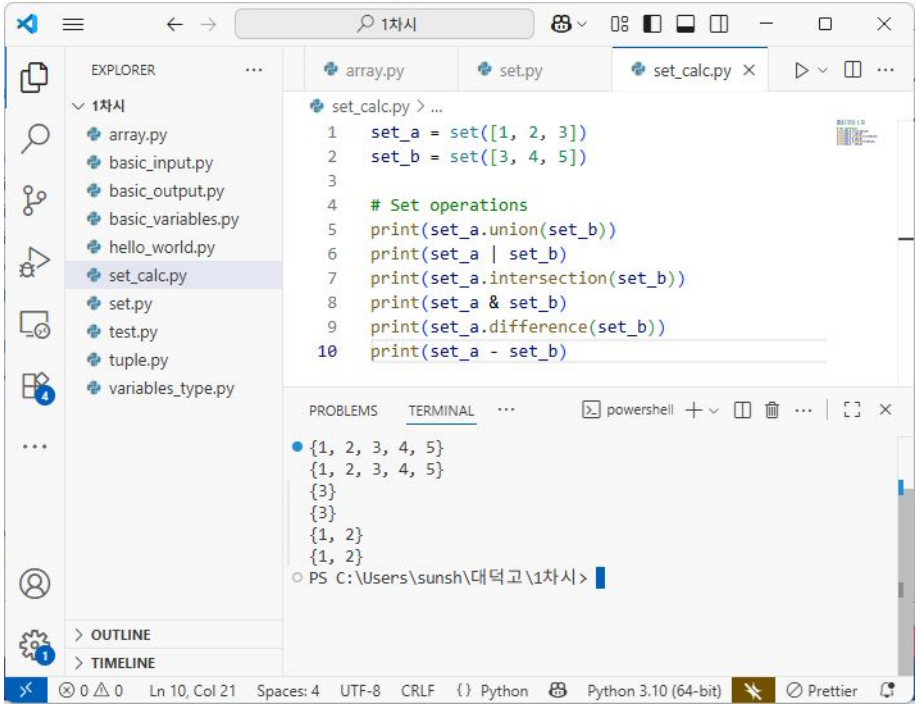
print({출력할 내용}, end=' ') 로 끝에 enter를 출력하지 않아도 됨

집합(set)



일반적으로 수학에서 배우는 집합과 같음

집합(set)



합: .union() or |

교집합: .intersection() or &

차: .difference() or -

사전(dictionary)

연관된 이름(key)과 값(value)를 쌍으로 저장하는 자료구조

(ex. 메뉴 이름과 가격, 학생 이름과 출석 여부)

딕셔너리를 선언하는 두 가지 방법

```
foodPrice = {"ramen": 10000,
             "chicken": 20000,
             "pizza": 30000}
```

```
foodPrice = dict(ramen=10000, chicken=20000, pizza=30000)
```

사전(dictionary)

딕셔너리의 key는 unique함, value는 중복도 허용.

key가 dictionary 안에 존재한다면, key를 사용해 key와 쌍을 이루는 value를 가져올 수 있음.

```
print(foodPrice["pizza"])
print(foodPrice.keys)
print(foodPrice.values)
```

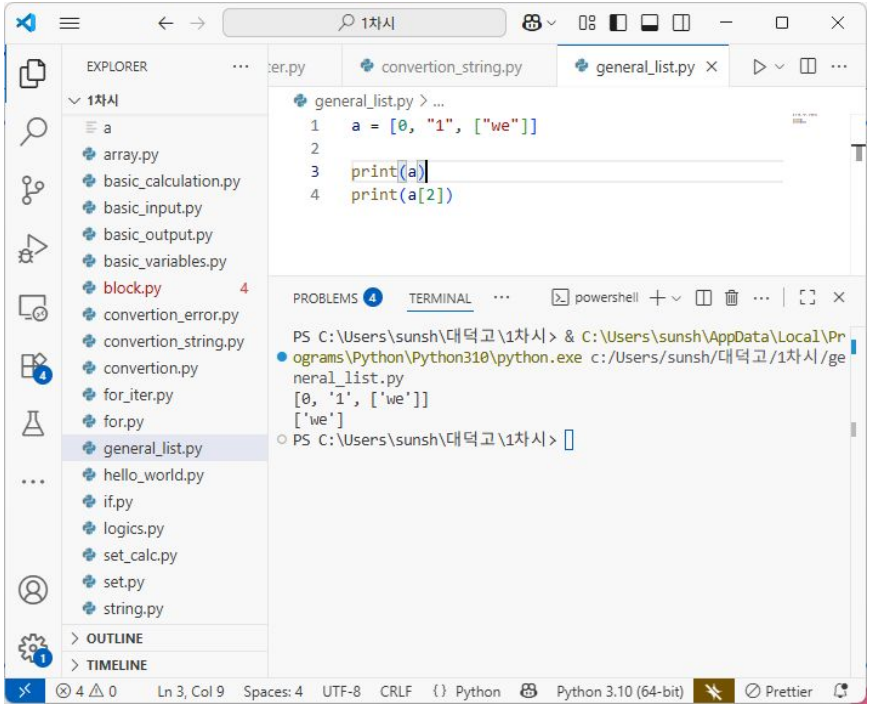
```
# let's modify the price
foodPrice["chicken"] = 22000

#let's add more food to the dictionary
foodPrice["rice"] = 1000

# let's remove pizza
del(foodPrice["pizza"])

# what happens if we ask something new?
print(foodPrice["dume"]) # Key Error!
```

리스트에는 어떤 원소가 들어갈까?

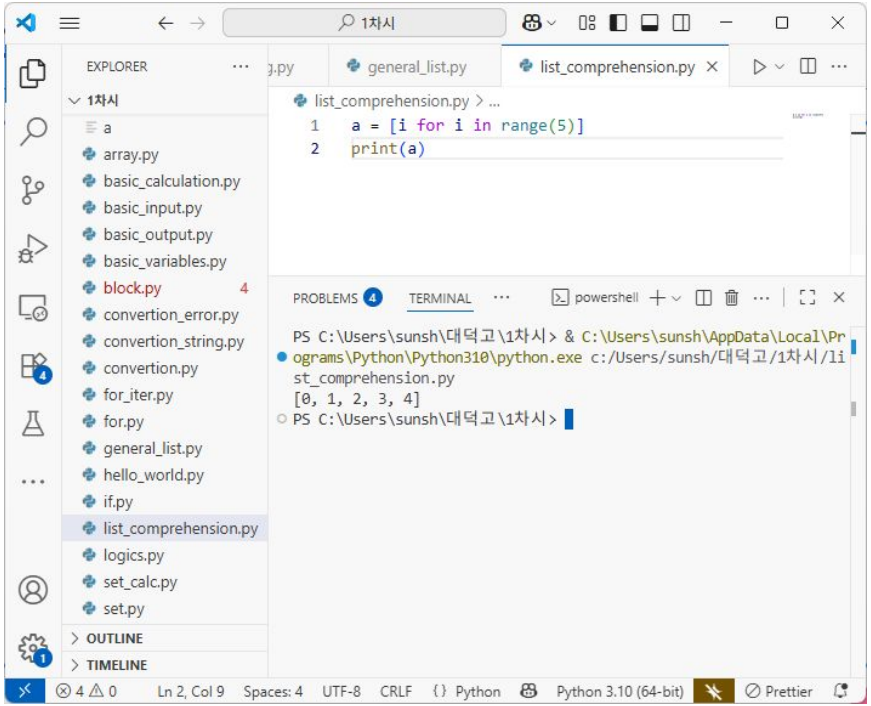


리스트에 들어가는 원소의 type이 달라도 된다.

리스트에는 중첩으로 다시 리스트가 들어갈 수 있다(set, dict...등도 동일)

list에 저장될 수 있는 원소의 종류는 object타입이기 때문!

list comprehension



리스트 원소를 간단한 for문으로 생성이 가능하다.

Class

클래스란 관련된 함수(메서드), 변수 등을 한 곳에 모아놓은 설계도.

예를 들어, Human이라는 class는 이름, 생일 등의 변수를 가질 수 있고,

밥 먹기, 코딩하기 같은 행동을 담당하는 함수 (메서드)를 가질 수 있음.

클래스는 class {클래스명} 으로 생성할 수 있음, 안에서 변수를 선언하거나 함수 (메서드) 를 추가할 수 있음

```
class Human:
    pass
```

Class 인스턴스, 생성자와 소멸자

클래스 자체는 설계도(타입)이라, 이걸 기반으로 실제 구현체를 만들어야 사용할 수 있음.

이 구현체를 인스턴스라고 부르며, 다음과 같이 만들 수 있음.

```
class Human:
    def __init__(self):
        print("태어났습니다")

    def __del__(self):
        print("죽었습니다")

seunghyun = Human()
```

생성자는 클래스로부터 인스턴스를 만들 때 자동

class 안의 __init__ 메서드로 생성자 정의 가능.

소멸자는 인스턴스 객체가 소멸할 때 자동 호출,

__del__ 메서드로 소멸자 정의 가능.

Class 변수와 instance 변수

class 변수는 클래스 내부에서 선언된 변수, 모든 인스턴스가 공유

instance 변수는 self. 로 시작하는, 인스턴스 별로 가지고 있는 특징적 변수 (속성!)

```python
class Human:
    population = 0 # class 변수

    def __init__(self, name):
        self.name = name # instance 변수
        self.age = 1
        print(f"{name}이(가) 태어났습니다")
        Human.population += 1

    def birthday(self):
        self.age += 1
        print(f"{self.name}이(가) {self.age}살이 되었습니다")

    def __del__(self, age):
        print(f"{self.name}이(가) {self.age}살에 죽었습니다")
        Human.population -= 1
```

클래스 변수는 클래스명.클래스변수명 으로

인스턴스변수명으로

```python
doyeon = Human("Doyeon")
doyeon.birthday()
print(f"name: {doyeon.name}")

seunghyun = Human("Seunghyun")
seunghyun.birthday()

print(Human.population)
```

실습

1부터 25를 5 * 5 사각형 모양의 배열에 넣어보자

물건의 이름과 가격을 담은 딕셔너리를 이용해서, 물건의 이름을 입력하면 가격을 출력해주는 프로그램을 작성하기

학생의 이름과 학번, 성적을 저장하는 클래스를 만든 다음, 3명의 학생 인스턴스를 생성하고 각 학생의 성적에 따라 A(>90)/B(>80)/C(>70)/D(이외)로 grade를 산출하는 프로그램을 작성하기